Mini-Project 1: Story Language Modeling with nanoGPT

Mayukh Das - U7965027 - COMP4680/ENGN8650, ANU

Task 1: Tuning Hyperparameters and Training Base Config on ROCStories

Data Processing

The full ROCStories dataset contains 98,161 stories split into train, validation and test sets of '78,528', '7,853' and '19,633' stories respectively. The stories are tokenized using tiktoken GPT-2 which has a vocabulary size of 50,257, an average five-sentence story of 44 words translates to approximately 51 tokens. The maximum story length in the full dataset was 108 tokens, hence 'block_size' was set as 128 to ensure every story fits within a single context window. Stories were concatenated sequentially and separated by the **<|endoftext|>** token, so the model can distinguish where one story ends, and another begins. The resulting training set is approximately 3.7M tokens. This is far below what the Chinchilla scaling law recommends, which is about 600M tokens for a 30M parameter model. This severe data deficit is the central constraint shaping all my experimental findings: the model cannot utilise its parameter budget over sufficient data, making every architectural and training choice a question of efficiency rather than capacity.

Training

My baseline follows a near baseline config of nanoGPT configuration, and the model was trained without any pretrained weights as per the assignment spec of 7 layers, 6 heads and 384 embeddings. The model was trained using a decoder-only transformer from scratch using Karpathy's nanoGPT. The model was trained on an L4 GPU on Google Colab taking about 44 minutes. **Appendix 2 demonstrates a full list of the exact hyperparameters I used for the run**. The default learning rate of 1e-3 caused overfitting and was dropped to 2.5e-4 to extend the effective training window, thus allowed the cosine learning rate schedule to decay smoothly over 12,000 steps without the model collapsing into memorization early. 'Warmup iters' was set to 500 to ensure, the early noisy gradients and initialization weights don't affect the initial training where loss reduction is highest. Block Size was set to 128, to reduce cross story noise since max story length was 108 tokens. Batch size was set at 128, to increase model context and give more stable gradients, which is especially important when training data is scarce. Dropout was set at 0.2 and weight decay 0.2 to prevent overfitting on small data. The main val loss reduction was seen by increasing training iters to 12000, giving the model more time to converge, and stay in a higher LR zone for longer.

The training and validation loss curves reveal three distinct phases. In the first 1,000 steps, loss drops sharply from 10.9 to roughly 4.0 as the model moves from random initialization to basic language patterns. We observe the 500-step warmup prevents unstable updates during this critical phase. Between steps 1,000 and 6,000, both train and validation curves descend steadily reflecting the same behaviour, indicating healthy generalization. After step 6,000, val loss plateaus near 3.30 while train loss continues falling to 2.37 by step 12,000 - a train to validation gap of approx. 0.93 at convergence. The widening gap can be explained as mild overfitting rather than catastrophic divergence, which the dropout and weight decay successfully contain. **The full training and validation loss curves are shown in Appendix 1.**

With limited data, the learning rate and schedule had a significant impact on training behaviour. Higher learning rates (e.g. 1e-3) caused the model to converge quickly but overfit early, leading to worse final performance. Reducing the learning rate to 2.5e-4 allowed the model to train more slowly and generalize better. The cosine decay schedule further extended the effective training window, allowing the model to remain in a useful learning regime for longer. Increasing training iterations to 12,000 also improved validation loss, suggesting that with smaller datasets, slower and longer training is more effective than fast convergence.

Evaluation

The model achieved a test perplexity of 25.77 on the full 19,633-story test split confirming that the model generalizes reasonably well despite the data constraint mentioned above. The generated stories show reasonable five-sentence structure and coherent local flow, though common failure modes include repetition, semantic inconsistency, and topic drift. Based on the Qwen evaluation rubric used for final grading, outputs at temperature 0.8 are estimated to score approximately 2.5-3/5 – adequate, following the prompt and reaching a conclusion but lack creativity and naturalness. **Complete samples across three decoding settings are provided in Appendix 3.**

At 31.69M parameters the model is not capable of deep causal reasoning, instead focusing on learning sentence formatting without narrative coherence. ROCStories itself is crowdsourced with considerable noise, demonstrating bland and formulaic training stories set a ceiling on generation quality.

Task 2: Investigating Generation Quality in a Data and Compute Constrained Tiny Language Model

Training a 30M parameter model on 3.7M tokens is all about making design decisions prioritising efficiency over capacity. I tried to investigate features that would best increase model performance keeping in mind the small dataset size. I explored four main ideas: tokenizer and vocabulary design, modern architectural components, depth vs. width, and data strategy. I found conventional wisdom from large-scale training does not always transfer to small models, with some improvements that help big models actively hurt on small data, and that parameter budget and embedding table size shapes architectural choices more than depth or width alone.

Tokenizer and Vocabulary Design

The single largest performance gain across all experiments came not from architecture or training strategy, but from tokenizer choice. Replacing the tiktoken GPT-2 - vocab size 50,257, with the Mistral tokenizer - vocab size 32,000 reduced ppl from 28.62 to 23.71 on an otherwise identical model.

The mechanism behind this discovery is straightforward. With tiktoken, the embedding table alone consumes 50,257 × 384 = nearly 19.3M parameters, over 60% of the total 31.69M parameter budget. The freed parameters are redistributed to transformer layers, where they directly contribute to learning rather than sitting in an embedding lookup table. Switching to Mistral's 32k vocabulary reduces the embedding table to approximately 12.3M parameters, freeing nearly 7M additional parameters for depth and attention. The model effectively gets more compute per token maintaining the same total parameter count.

This result demonstrates the significant importance on tokenizer choice, particularly in parameter-budget-constrained regimes. However, the Mistral tokenizer is incompatible with the provided eval.py script in the assignment which requires tiktoken encoding, so all Task 3 submissions use tiktoken. The Mistral results are reported here as a research finding only.

Architectural Components

To study the contribution of modern architectural components drawn from the LLaMA architecture, I isolated the contribution of three modern - RoPE positional encoding, RMSNorm, and SwiGLU activation, running them on P10n baseline (PPL 25.33), applying one component at a time.

Rotary Positional Embedding (RoPE) replaced standard learned positional embeddings and gave the largest improvement, reducing PPL from 25.33 to 24.69. Unlike learned embeddings which consume parameters in the embedding table, RoPE encodes position directly into the attention computation at no additional parameter cost, which led to a pure efficiency gain under my budget constraint.

Replacing LayerNorm with RMSNorm, gave a modest but consistent improvement from 25.33 to 25.01. RMSNorm removes the re-centering operation from standard LayerNorm, reducing computation and stabilizing training slightly on small data.

SwiGLU replacing GELU hurt performance, pushing PPL from 25.33 to 25.62, diverging after 6500 iterations. SwiGLU introduces a gating mechanism that requires sufficient data to learn meaningful gate values, which at 3.7M tokens, the model simply cannot support this added complexity.

Combining RoPE and RMSNorm gave PPL 24.67, showing the two improvements are complementary, but not additive. Thus, on small data only two of the three LLaMA components transfer effectively, SwiGLU being the component that requires scale to justify its complexity. **Architectural components results are summarised in Appendix 4.**

Depth vs. Width

Kaplan et al. (2020) demonstrates that within reasonable limits, model performance depends very weakly on architectural hyperparameters such as depth vs. width. Staying within the stipulated maximum parameters limit of 32M, I systematically varied depth and embedding size to maximise parameter size and tested various combinations of n_layers, n_heads and embeddings, to find optimal hyper parameters combinations.

In my experimentation, depth consistently wins at matched parameter count - P12a with 16 layers beats P12c 11 layers by 0.28 PPL points with nearly identical parameter counts. However, the overall winner is P14a with 20 layers, 272 embedding at 25.12, demonstrating more parameters and more depth is a clear winner. This suggests that under such constraint, parameter count, and more depth are superior and deeper models make better use of additional parameters than wider ones.

This partially aligns with Kaplan et al, parameter count dominates, but in my case, depth findings contradict their finding that depth vs. width is neutral. This suggests, on smaller data, depth helps the model understand features more robustly - each layer refines the previous layer's understanding rather than adding new parallel features that the model lacks sufficient data to learn. Width effectively wastes capacity on small datasets. **Full depth vs. width experimentation results are in Appendix 5.**

Data Strategy

We explored five data augmentation strategies to the ROCStories training set, since dataset size was an established serious constraint. Most augmentation approaches hurt performance except fine-tuning on training dataset with quality-filtration.

2,000 good quality ROCStories-like stories generated across multiple chatbots hurt the PPL and Qwen rating, as the data distribution mismatch may have confused the model further. TinyStories pretraining performed the worst, despite being a massive dataset which could potentially solve the limited data issue, as simplified children's vocabulary may have been too different to ROCStories for fine-tuning to recover it. Gemini fine-tuning revealed a PPL/quality disconnect - perplexity worsened but estimated Qwen rose to 4 or 5, suggesting richer training stories can improve generation naturalness, though at the cost of test distribution fit. Quality-filtered fine-tuning on ROCStories itself was the only strategy that improved both metrics, reducing PPL from 25.33 to 24.92 by removing noisy and malformed stories while staying close to the original sentence structure.

The consistent finding across all augmentation experiments is that distribution match supersedes data quantity. Data that doesn't follow the testing dataset structure, consistently worsens perplexity, regardless of quality or size. **Complete data augmentation results are in Appendix 6.**

Final Model

My Task 3 submission is P15ab, a 20-layer, 8-head, 256-embedding nanoGPT with RoPE positional encoding and RMSNorm, trained for 12,000 steps at lr=2.5e-4, achieving a test perplexity of 24.67. However, P16ab (20/8/272 + RoPE + RMSNorm, PPL 24.84) produces qualitatively better stories despite higher perplexity, demonstrating PPL vs quality disconnect observed in the data strategy experiments. **Full configuration and training curves for the submission model are in Appendix 7 and 8. Generated samples are in Appendix 9.**

Decoding with a temperature of 0.65 and top_k=50 produced the best outputs across different prompts, giving coherent five-sentence stories with clear endings. At lower temperatures (0.6), the model became too deterministic and often repeated the same outputs across samples. At higher temperatures (0.8 and above), the stories started to lose coherence, with noticeable topic drift and inconsistent details. Overall, 0.65 provided a good balance between maintaining structure and allowing enough randomness for more natural variation.

References

1. Karpathy, A. (2022). nanoGPT. GitHub. https://github.com/karpathy/nanoGPT

2. Kaplan, J., et al. (2020). Scaling laws for neural language models. arXiv:2001.08361

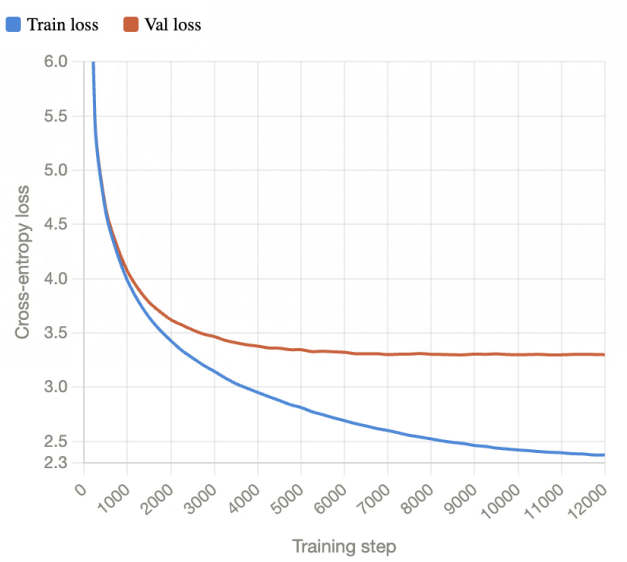
3. Hoffmann, J., et al. (2022). Training Compute-Optimal Large Language Models. arXiv:2203.15556

4. Eldan, R., & Li, Y. (2023). TinyStories: How Small Can Language Models Be and Still Speak Coherent English? arXiv:2305.07759

5. Jiang, A., et al. (2023). Mistral 7B. arXiv:2310.06825

Appendix

Appendix 1 – Train and Validation Loss plot for Task 1 Model



Appendix 2 – Task 1 Model Config

Parameter	Value
n_layer	7
n_head	6
n_embd	384
Batch Size	128
Block Size	128
Learning Rate	2.5×10^{-4}
Minimum Learning Rate	2×10^{-5}
Max Iterations	12,000
LR Decay Iterations	12,000
Warmup Iterations	500
Dropout	0.2
Weight Decay	0.2
$Beta_1$	0.9
$Beta_2$	0.99
Gradient Clipping	1.0
Data Type	bfloat16
Tokenizer	GPT-2 BPE (tiktoken)
Vocabulary Size	50,257
Number of Parameters	31.69M
Test Perplexity (PPL)	**25.77**

Appendix 3 – Task 1 Story Generation

Temperature =0.6, Top_k=20 (conservative)

Story 1: "Sarah wanted to bake a cake for her friend's birthday. She wanted to make a cake. She bought all the ingredients. She mixed it together. She made it the cake."

Story 2: "Sarah wanted to bake a cake for her friend's birthday. Sarah decided she would make a cake with a cherry pie. Sarah went to the store and bought the ingredients. Sarah started to bake the cake. When she tasted the cake, she was shocked to see that it was not the same!"

Temperature =0.8, Top_k=50 (balanced)

Story 1: "Sarah wanted to bake a cake for her friend's birthday. She wanted some blueberry. Sarah baked the cake for her friend. When it came time for the cake, Sarah took a bite. Sarah tasted the blueberries and was happy."

Story 2: "Sarah wanted to bake a cake for her friend's birthday. Sarah decided she would knit a cake with a lot of time. Sarah went to the store to buy an all-nighter. When she got to class, she started to finish her cake. Sarah's cake turned out beautiful and her friend never came back."

Temperature =1.2, Top_k=200 (creative)

Story 1: "Sarah wanted to bake a cake for her friend's birthday. She wanted some sausage but didn't have enough money so she called her friend Tara. Tara made a list of what she needed and saved her money! She bought a great gift and they came home for her. They watched TV and put the dough on the oven afterwards."

Story 2: "Sarah wanted to bake a cake for her friend's birthday. Sarah always wanted one for a cake with a chip. She decided to try making a cake. Many local family came out with almost no cake. Sarah's cookie was not the best."

Appendix 4 – Architecture Component Experiments

Model	Change	PPL
P10n	Baseline	25.33
P15b	RoPE	24.69
P15a	RMSNorm	25.01
P15c	SwiGLU	25.62
P15ab	RoPE + RMSNorm	24.67

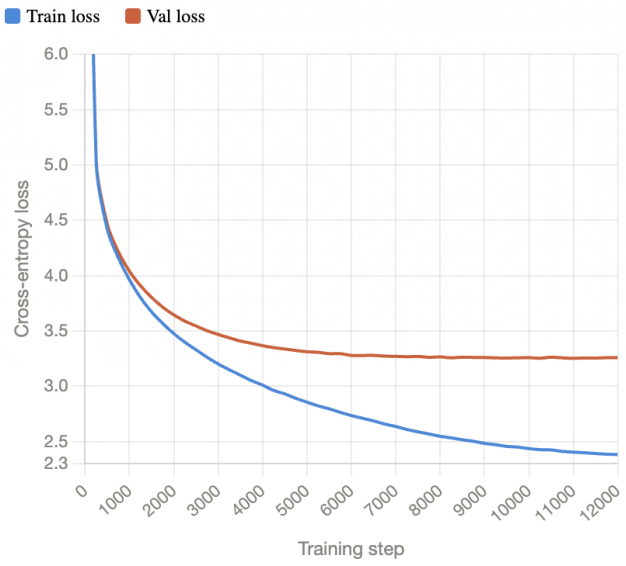
Appendix 5 – Depth vs Width Experiments

Model	n_layer	n_head	n_embd	Params	PPL
P10k	7	6	384	31.69M	25.77
P12c	11	8	336	31.80M	25.45
P12a	16	8	296	31.71M	25.17
P11c	20	8	256	28.60M	25.44
P11d	20	10	250	28.55M	25.41
P10n	20	8	256	28.60M	25.33
P14a	20	8	272	31.53M	25.12

Appendix 6 – Data Augmentation Experiments

Strategy	Data	PPL	Qwen (est.)	Finding
Baseline	ROCStories (78k)	28.62	~2/5	-
+2k synthetic	ROC + mixed synthetic (Gemini, ChatGPT, Perplexity)	30.35	-	Degraded performance
+20k synthetic	ROC + Gemini API 20k	30.16	-	Degraded performance
TinyStories pretrain	TinyStories → ROC ft	53.26	-	Performed poorly
Gemini ft	P10n + Gemini API 20k ft	27.16	~4/5	PPL/quality tradeoff
Filtered ft	P10n + filtered ROC 75k	**24.92**	~3/5	best data_augmentation result

Appendix 7 – Train and Validation Loss plot for Task 3 Model



Appendix 8 – Task 3 Model Config

Parameter	Value
n_layer	20
n_head	8
n_embd	256
Batch size	128
Block size	128
Learning rate	2.5×10^{-4}
Minimum learning rate	2×10^{-5}
Max iterations	12,000
LR decay iterations	12,000
Warmup iterations	200
Dropout	0.2
Weight decay	0.2
$Beta_1$	0.9
$Beta_2$	0.99
Gradient clipping	1.0
Data type	Bfloat16
Architecture	RoPE + RMSNorm
Tokenizer	GPT-2 BPE (tiktoken)
Vocabulary size	50,257
Number of parameters	28.60M
Test perplexity (PPL)	24.67

Appendix 9 – Task 3 Story Generation

Temperature = 0.6, Top-k = 20 (conservative)

Story 1: Sarah wanted to bake a cake for her friend's birthday. She wanted to bake a cake. Sarah went to the store to buy the cake. Sarah picked out a cake that she really liked. Sarah got the cake for her friend's birthday.

Story 2: Sarah wanted to bake a cake for her friend's birthday. She drove to the store and bought all the ingredients. Sarah then went home to bake the cake. She was excited to bake the cake. Sarah baked the cake and it was delicious.

Temperature = 0.65, Top-k = 50 (balanced)

Story 1: Sarah wanted to bake a cake for her friend's birthday. She wanted to bake a cake. Sarah went to the store to buy the cake. Sarah picked out a cake that she really liked. Sarah got the cake for her friend's birthday.

Story 2: Sarah wanted to bake a cake for her friend's birthday. She drove to the store and bought all the ingredients. Sarah then went home to bake the cake. She was pleased to see it was ready to be baked. She served the cake to her friend at the party.

Temperature = 0.8, Top-k = 50 (creative)

Story 1: Sarah wanted to bake a cake for her friend's birthday. She wanted some chocolate but her mom was out of town so she called her mom. Her mom made her a pumpkin cake and put it in the oven. She looked at the pie and she found the crust. Sarah was glad she made the cake and she ate it.

Story 2: Sarah wanted to bake a cake for her friend's birthday. Sarah gathered all the ingredients together and made a cake. Sarah baked the cake and served it to her friend. The friend loved the cake and ate the cake. Sarah didn't feel so bad that she ate the cake.